# Disallow Binding a Returned Glvalue to a Temporary


| | |
|---|---|
| Document #: | P2748R2 |
| Date: | 2023-09-14 |
| Project: | Programming Language C++ |
| Audience: | Evolution |


| | |
|---|---|
| Reply-to: | Brian Bi <bbi10@bloomberg.net> |


# Contents



# § 1 Revision history

## § 1.1 R2

— Removed library wording and added a carve-out to core wording

— Added discussion of implementation experience

## § 1.2 R1

— Added another motivating example

— Added discussion of unevaluated contexts

— Added library wording

# § 2 Introduction

The following code contains a bug: The code initializes a reference from an object of a different type — the programmer has forgotten that the first element of the pair is `const` — and creates a temporary. As a result, the reference `d_first` is always dangling:

```
struct X {
    const std::map<std::string, int> d_map;
    const std::pair<std::string, int>& d_first;

    X(const std::map<std::string, int>& map)
        : d_map(map), d_first(*d_map.begin()) {}
};
```

Luckily, the above code is actually ill formed (§ [class.base.init]p8 of the Standard[1]). Nonetheless, valid code can contain essentially the same bug:

```
struct Y {
    std::map<std::string, int> d_map;

    const std::pair<std::string, int>& first() const {
        return *d_map.begin();
    }
};
```

This code is valid, although compilers might warn. Like the first code snippet in this paper, this one always produces a dangling reference. We should make this code likewise ill formed.

A colleague recently reported another example. A program appeared to be accessing memory that was unsafe to access. The bug was ultimately caused by the following function returning by reference though it should not have:

```
const std::string_view& getString() {
    static std::string s;
    return s;
}
```

Finding this bug was difficult yet would have been easy if the `return` statement were simply ill formed.

# § 3 Background

In [CWG1696], Richard Smith pointed out that, though binding a reference member to a temporary in a *mem-initializer* was explicitly called out in the Standard as one of the cases in which the lifetime of the temporary is *not* extended to the lifetime of the reference, no

corresponding wording was offered for the case in which the expression that produces the temporary is supplied by a default member initializer.

Initially, the proposed resolution simply resolved the inconsistency in favor of explicitly specifying that *brace-or-equal-initializers* behave the same way as *mem-initializers* (i.e., neither extends lifetime). However, at the Issaquah meeting in 2014, making both ill formed was suggested. CWG appears to have accepted this suggestion without controversy. (At the Urbana-Champaign meeting later that year, Issue 1696 was given DR status.)

This change was so uncontroversial because binding a reference to a temporary, when the reference will outlive the temporary and become dangling as soon as the full-expression completes, is always a bug. In some simple cases, a novice programmer might not understand that a temporary must be materialized when binding a reference to a prvalue. On the other hand, the examples given in the introduction represent code that experienced C++ developers can easily write.

# § 4 Proposal

The dangling reference created by `X`'s constructor is always a bug, and the same is true for the dangling reference created by `Y::first`. In fact, one can imagine some obscure situations in which binding a reference member to a temporary in a *mem-initializer* could be useful to cache the result of an expensive computation, which could then be used by later *mem-initializers* and within the *compound-statement* of the constructor. In contrast, when binding a returned glvalue to a temporary, even such obscure, limited applications seem nonexistent.

I propose, therefore, to make binding a returned glvalue to a temporary likewise ill formed.

Note that recent versions of Clang, GCC, and MSVC all issue warnings that explain the creation of the dangling reference. The availability of such warnings raises the question of whether programmers should simply use compiler flags to convert those warnings into errors, thus obtaining all the benefits of this proposal with no need for a language change. However, at least in Clang and GCC, the warnings have false positives, which (as discussed in Section 5) occur because they are less narrowly scoped than this proposal. More broadly, compiler warnings are no substitute for language rules because the warnings lack formal specification and are not portable.

# § 5 What about unevaluated `return` statements?

At the February 2023 meeting in Issaquah, the EWG asked for improved wording related to unevaluated contexts. However, no such thing as an unevaluated `return` statement

exists (at least from the core language point of view; see Section 6 for discussion of the library).

Section 6.3 [basic.def.odr]p3 of the Standard defines a conversion as *potentially evaluated* unless it is "an unevaluated operand, a subexpression thereof, or a conversion in an initialization or conversion sequence in such a context." Because a `return` statement is not an expression statement, the only kind of expression a `return` statement can appear within is a lambda expression, but the statements in the body of a lambda expression are not subexpressions of the lambda expression (§ [intro.execution]p3.3), so even if the lambda expression is unevaluated, the statements in its body are still potentially evaluated.

This definition is not simply a technicality but follows from the very nature of function definitions in C++. When the body of a lambda expression is instantiated, a function definition is created, and a function definition created by an instantiation triggered from an unevaluated context is no different from any other definition of the same function. In particular, that function may be ODR-used at some later point, but the compiler is not expected to instantiate it a second time since the instantiation from an unevaluated context is as good as any other instantiation. Attempting to carve out a narrow exemption that applies exclusively to `return` statements appearing lexically within lambda expressions that are not potentially evaluated would, therefore, fail to actually prevent such `return` statements from being evaluated at run time.

For this reason, my proposal does not include carving out an exemption for lambdas in unevaluated contexts. This exclusion raises the question of whether the proposal would disallow some useful metaprogramming techniques.

[P0315R2] discusses two use cases for lambdas in unevaluated contexts. In both use cases, the lambda is used only for the signature of its function call operator. In such cases, the `return` statement in the lambda could be eliminated, and the lambda could be given a trailing return type instead. Rewriting the code in this fashion is annoying but will be necessary in only the tiny fraction of cases where lambdas in unevaluated contexts currently contain `return` statements that would create dangling references if they were to be evaluated. The benefits of this proposal outweigh the inconvenience that would be inflicted in those very few cases.

As evidence that this situation is almost nonexistent, consider that recent versions of Clang and GCC do not distinguish `return` statements appearing in unevaluated lambda expressions from those that appear in any other function and will issue a warning even in cases such as the following:

```
std::string_view sv;
decltype ( [] () -> const std::string_view& {
    static std::string s;
    return s;
} () ) svr = sv;
```

I searched the Clang and GCC bug trackers for reports of false positives for the `-Wreturn-stack-address` and `-Wreturn-local-addr` flags, respectively. Some false positives were

reported, but they generally appear to be related to these warnings going far beyond the set of situations that this paper proposes to make ill formed; the warnings perform a flow analysis to check whether a returned pointer value might have been derived directly or indirectly from the address of a temporary or an automatic variable. GCC bug 100403 and Clang bug 44003 are representative of this class of bugs. I found no issues in which a user opined that the warning should not fire because the `return` statement was in a lambda expression in an unevaluated context.

# § 6 Need for changes to `std::is_convertible`

As pointed out at the February 2023 meeting in Issaquah, the current definition of the `std::is_convertible` type trait (21.3.7 [meta.rel]p5) depends on the well-formedness of a `return` statement but is intended to detect implicit convertibility in general. For this reason, this proposal must ensure that the meaning of `std::is_convertible` does not change; for example, `std::is_convertible_v<int, const double&>` should continue to be `true`.

Since, as discussed previously, no such thing as an unevaluated `return` statement exists, giving a blanket exemption for such nonexistent entities is an impractical solution to this problem. Instead, three possible approaches present themselves.

1. Add a special exception only for the Standard Library.
2. Re-express `std::is_convertible` in terms of a piece of code that does not contain a `return` statement.
3. Re-express `std::is_convertible` in terms of the core language concept of implicit convertibility.

The second approach is feasible if we assume (as current implementations do) that the `To` type must be destructible. In that case, `std::is_convertible_v<From, To>` is true if all the following conditions are met.

— `To` is not an array type.

— `To` is not a function type.

— If either `To` or `From` is *cv* `void`, then so is the other.

— If neither `To` nor `From` is *cv* `void`, then `requires (void (*f)(To), From&& arg) { f(static_cast<From&&>(arg)) }` is `true`.

However, since [LWG3400] is unresolved, the specification of `std::is_convertible<From, To>` could possibly be changed to exclude consideration of the destructor (which appears to imply that the implementation will require compiler magic). The second approach would therefore assign an interpretation to the current specification of `std::is_convertible` that would be contentious in the LWG.

Furthermore, the effort that would be spent in the LWG on codifying this approach would be wasted if the LWG later decided to exclude the destructor. I am, therefore, not proposing adopting this approach at this time.

The third approach also suffers from similar issues. Implicit convertibility is defined by § [conv.general]p3 in terms of the well-formedness of a hypothetical declaration employing copy-initialization. Plainly, such a declaration is not well-formed if the destination type is not destructible, so taking this approach assumes a particular disposition for [LWG3400]. Expressing `std::is_convertible` in terms of the existence of an implicit conversion sequence (as defined by § [over.best.ics.general]) would assume the opposite disposition, while also subjecting the library to the unresolved issue that is the subject of [CWG2525].

Therefore, I propose the first approach. The previous revision of this paper proposed library wording that exempted the `std::is_convertible` trait from the proposed core wording. Following the EWG's feedback in Varna, the carve-out has been moved to core.

# § 7 Implementation experience

I have built a patched version of Clang 16.0.6 that implements the change proposed in this paper. Using the patched Clang, I successfully built Clang itself, which contains an estimated 3.7 million lines of C++ code in the `llvm` and `clang` subdirectories of the `llvm-project` repository ([LLVM]). There were 13 failed tests, which can be divided into the following categories.

— **Tests that verify that Clang issues a warning for a return statement that binds a returned reference to a temporary fail because the warning is now an error** — Each such test can be replaced by a test that verifies the issuance of an error and, optionally, a test in which the temporary is returned through an indirect path for which Clang is still expected to issue a warning.

— **Tests that verify that a function compiles to a particular LLVM bitcode sequence, e.g., functions that return references to temporaries** — If such functions are made ill formed as this paper proposes, such tests can simply be deleted.

— **Tests that verify that Clang's constant evaluator diagnoses a read from a temporary whose lifetime has ended** — These tests can be amended so that the function that returns the reference does so through an indirect path that does not trigger an error under this paper's proposed wording but should still cause the constant evaluation to fail.

— **Tests that use the return statement only to check that binding a particular reference to a particular temporary is well formed** — These tests can be changed so that they bind a local reference rather than a returned reference.

I also successfully built Bloomberg's BDE repository ([BDE]), including all tests (1.7 million lines of C++ code) and Chromium ([Chromium]), comprising 39 million lines of C++ code. These estimates were generated using David A. Wheeler's SLOCCount

([SLOCCount]). These results suggest that the change proposed by this paper is unlikely to cause many compilation errors in existing code that has already been reviewed and successfully deployed.

# § 8 Wording

The proposed wording is relative to [N4958].

Strike p6.11 in § [class.temporary]:

> — ~~The lifetime of a temporary bound to the returned value in a function `return` statement (8.7.4) is not extended; the temporary is destroyed at the end of the full-expression in the `return` statement.~~

Insert a new paragraph, 6, at the end of § [stmt.return]:

> In a function whose return type is a reference, other than an invented function for `std::is_convertible` ([meta.rel]), a `return` statement that binds the returned reference to a temporary expression ([class.temporary]) is ill-formed. [*Example 2:*
>
> ```
> auto&& f1() {
>     return 42;  // ill-formed
> }
> const double& f2() {
>     static int x = 42;
>     return x;   // ill-formed
> }
> auto&& id(auto&& r) {
>     return static_cast<decltype(r)&&>(r);
> }
> auto&& f3() {
>     return id(42);  // OK, but probably a bug
> }
> ```
>
> — *end example*]

(*Note:* See [CWG GitHub issue 200] regarding a possible issue with the above wording.)

# § 9 References


[BDE]
https://github.com/bloomberg/bde

[Chromium]
https://www.chromium.org/Home/

[CWG GitHub issue 200] Brian Bi. 2022-12-16. Missing definition of “temporary expression.”
https://github.com/cplusplus/CWG/issues/200

[CWG1696] Richard Smith. 2013-05-31. Temporary lifetime and non-static data member initializers.
https://wg21.link/cwg1696
[CWG2525] Jim X. 2021-09-25. Incorrect definition of implicit conversion sequence.
https://wg21.link/cwg2525
[LLVM]
https://github.com/llvm/llvm-project/
[LWG3400] Jiang An. 2020-02-10. Does `is_nothrow_convertible` consider destruction of the destination type?
https://wg21.link/lwg3400
[N4958] Thomas Köppe. 2023-08-14. Working Draft: Programming Languages – C++.
https://isocpp.org/files/papers/N4958.pdf
[P0315R2] Louis Dionne. 2017-06-18. Lambdas in unevaluated context.
https://wg21.link/p0315r2
[SLOCCount]
https://dwheeler.com/sloccount/

---

1. All citations to the Standard are to working draft N4958 unless otherwise specified.↩